RUTURAJ SONKAMBLE · SOFTWARE ENGINEER (OPERATIONS TEAM), LONDON

Four pieces of manual work I'd love to delete at Corgi

Your JD says the Ops team builds three things: the internal tools underwriting, claims and policy admin live in, the platform policyholders log into, and the integrations that connect both to the rest of the market. So I picked one idea for each, plus the plumbing underneath.

WHO IT'S FOR	THE MANUAL WORK TODAY	WHAT I'D BUILD
Underwriting	Someone reads three loss runs that disagree about the same claim and decides which number to trust.	**1. EvidenceGate**: every figure keeps its source, conflicts get flagged, underwriting gets one clean snapshot with receipts.
Claims	A driver calls from the side of the road, and the claims team rebuilds the story from phone notes and loose photos.	**2. Five-minute FNOL**: a phone link that turns "I had an accident" into a structured claim file.
Policy admin & policyholders	"Can you send a certificate to my new shipper?" arrives by email and gets done by hand.	**3. Policy Wallet**: COIs, vehicles and drivers in one place, with the common requests a few taps away.
Integrations	A timeout during quote-to-bind leaves everyone unsure whether the policy exists, and a retry can bind it twice.	**4. BindSafe**: recover the true state before retrying anything, and prove it with failure tests before a partner goes live.

HOW I'D APPROACH ALL FOUR

- **Build on top of what exists.** None of these replace Corgi's core systems. They sit next to them and remove the hand-work.
- **AI proposes, a person or a rule decides.** AI is great at reading documents and drafting. Anything that changes a policy, a price or a claim stays with a rule or a licensed human.
- **Small first version, shipped fast.** Each idea has a "week one" slice that's useful on its own.

These are directions based on public information about Corgi, not claims about gaps in your current software. I'd validate each workflow with the team before building against real systems or data.

UNDERWRITING

1. EvidenceGate

Don't pick a number. Keep the receipts, show the conflict, then decide.

> Three documents mention the same claim. Loss run A says **$42K**, loss run B says **$67K**, and a later update says **$75K**. Today someone has to notice that, figure out they're the same claim, and pick one. If nobody notices, the wrong number quietly flows into pricing.

```
Claim #1842
  $42K   Loss run A    received 12 Jan   p.3
  $67K   Loss run B    received 02 Feb   p.1
  $75K   Carrier note  received 19 Feb   p.2
  ------------------------------------------
  Current value: $75K   reason: latest authoritative source
  Affects: incurred losses -> underwriting input
```

WHAT I'D BUILD

- An evidence store: every extracted fact keeps its document, page, received time, and which earlier value it replaced.
- Claim matching across duplicate records and different claim numbers for the same loss.
- Conflict detection for changed reserves, policy dates, and fleet or driver details.
- A review screen showing the conflict, the evidence, and what downstream fields it affects.
- A clean underwriting snapshot exported to the existing workflow.

WHERE AI FITS, AND WHERE IT DOESN'T

AI reads the documents and suggests matches. Deterministic rules decide which source is authoritative, and anything ambiguous goes to a person with the evidence already laid out.

WEEK ONE SLICE

Take one real submission, extract claim figures from its loss runs, and show every conflict side by side with sources. Even without automation, that's an hour of reading saved per file.

CLAIMS

2. Five-minute FNOL

The worst moment of a driver's week shouldn't also be the most paperwork.

> A truck driver has just had an accident. They're stressed, on the roadside, and the last thing they can do is find a policy number or write a careful report. Today that usually becomes a phone call, and the claims team rebuilds the story later.

```
"I had an accident."
   Are you safe?            -> Yes
   Where are you?           -> GPS captured
   Take 4 photos            -> camera opens
   What happened?           -> 30-second voice note
   Submit                   -> structured claim file created
```

WHAT I'D BUILD

- A mobile web page opened from a text message or QR code in the cab. No app to install.
- Secure policy and identity lookup before anything sensitive.
- Photos, location, time and a voice note turned into an editable written statement.
- One structured incident record for the claims team instead of a pile of messages.
- A status page so the driver knows what happens next.

WHERE AI FITS, AND WHERE IT DOESN'T

Speech-to-text and structuring the statement: yes. Coverage or liability decisions: no. Those stay with licensed adjusters. The goal is a complete file on day one, not a robot adjuster.

WEEK ONE SLICE

The phone flow plus a structured FNOL payload, tested end to end on a phone. It's easy to demo and has an obvious before and after.

POLICY ADMIN & POLICYHOLDERS

3. Policy Wallet

The five things a fleet manager actually needs, each a few taps away.

> A fleet manager lands a new shipper who wants proof of insurance today. They email Corgi, someone generates the certificate by hand, and everyone waits. Multiply that by every vehicle change, driver change and document request.

```
ACTIVE POLICY   Commercial Auto
  48 vehicles  |  53 drivers
  [ Send a COI ]  [ Add / remove vehicle ]  [ Documents ]  [ Report a claim ]

  Send a COI:  pick vehicle -> pick certificate holder -> confirm -> sent and tracked
```

WHAT I'D BUILD

- One-tap access to certificates and policy documents.
- Vehicle and driver lists pulled from the source of truth, never re-typed.
- Simple change requests (add a vehicle, update a driver) with clear status.
- An audit trail for every request and response.

WHY IT'S AN OPS WIN TOO

Every request a customer can finish themselves is one the policy admin team never has to touch. Endorsements that need review still go to a person, just with the details already filled in.

WEEK ONE SLICE

Self-serve COI generation for one policy type, with delivery tracking. It's the highest-frequency request and the easiest to measure.

INTEGRATIONS

4. BindSafe

A timeout is not an answer. Don't guess, recover the truth.

> A customer pays. The bind succeeds. Then the service crashes before the response comes back. The customer sees an error and clicks "Try again". A normal system might bind the policy twice, or charge twice, or leave the policy half-issued.

```
Payment   SUCCESS
Bind      UNKNOWN      <- response lost in a crash
Policy    UNKNOWN
          service restarts, reconcile with the source of truth
Bind      SUCCESS
Policy    POL-72931    <- the retry finds it instead of creating a second one
```

WHAT I'D BUILD

- Durable transaction state and idempotency keys for quote, payment, bind and issuance.
- An outbox, so events and webhooks go out exactly when the database says they should.
- Recovery workers that reconcile with the authoritative system before any retry.
- A small ops view for transactions stuck in "unknown", so a person sees them first.

THEN BREAK IT ON PURPOSE

Before a new partner or insurer integration goes live, run a failure suite against it: duplicate requests, a webhook arriving twice or late, payment succeeding while the bind response is lost, expired quotes, out-of-order events. The output is a pass/fail report:

FAIL duplicate bind possible under retry
PASS late webhook handled idempotently

Found in testing, not in production.

I've hit this exact class of bug before. In RaceLab, AI agents sharing a $100 limit broke it in 45 to 48 of 50 runs because they retried stale decisions; re-reading the true state before acting brought that to 0 of 50 (github.com/rajj28/racelab).

WHO'S WRITING THIS

Why I think I'd be useful on this team

- **Retries and duplicates are my day job.** At Concentrix I run the Go and AWS backend that queues and routes about 5,000 AI voice calls a day.
- **I've built tools that read insurance documents.** A retrieval system over real policy PDFs (ICICI Lombard, HDFC Ergo) that answers coverage questions with sources.
- **I care about whether results can be trusted.** My projects publish what failed next to what worked, and every number traces back to a recorded run.
- **I talked these ideas through with Jon Duffield** on your enterprise sales team when there wasn't an open role. This is the follow-through.

IF I JOINED, MY FIRST TWO WEEKS

- **Days 1 to 2:** sit with an underwriter, a claims handler and someone in policy admin, and write down every copy-paste I see.
- **Days 3 to 5:** ship the smallest tool that removes the most-repeated one. Your JD says production in week one, and I'd like to hold you to that.
- **Week 2:** measure the time it saved, then pick the next one. These four ideas are a starting list, not a plan I'd force onto the team.

Ruturaj Sonkamble · Pune, India · open to London (UK sponsorship needed)
github.com/rajj28 · ruturajsonkamble29@gmail.com